

FinSight

An agentic RAG analyst over public SEC filings

Project explanation: design, implementation, evaluation and lessons learned

Repository	github.com/AtharvaMusale/finsight
Project owner	Atharva Musale
Document date	30 September 2026
Scope	Steps 1 to 8 (ingestion to tracing and README), the live verification run, and the verifier fixes that followed it
Models	Claude Haiku 4.5 only (plan, compose, critic)
Corpus	AAPL, MSFT and NVDA 10-K and 10-Q filings, about three years

Contents

Executive summary	4
1. Introduction	6
1.1 Problem and goals	6
1.2 Non-goals	6
1.3 Design principles and how each is enforced	6
1.4 Technology stack	7
1.5 Project history	7
2. Data and ingestion	8
2.1 Sources and the SEC access policy	8
2.2 The corpus	8
2.3 Parsing and chunking	8
2.4 Embeddings and the hybrid index	8
2.5 DuckDB: filings, chunks and facts	9
2.6 Ingestion lessons	9
3. Architecture	10
3.1 System overview	10
3.2 The orchestrator graph	10
3.3 Interfaces	11
3.4 A request, step by step	11
4. Retrieval in depth	12
4.1 Hybrid search mechanics	12
4.2 Choosing alpha	12
4.3 Reranking: better ordering, worse coverage	13
4.4 Query planning: the router	13
5. The facts route: SQL and the calculator	15
5.1 Guarded text-to-SQL	15
5.2 The calculator and derived metrics	15
5.3 Composition and citation labels	15
6. The verifier	16
6.1 The three layers	16
6.2 The revise loop, caveats and outages	16
6.3 Known limits of the verifier	16

7. The MCP server	17
8. The A2A services	17
8.1 Delegation is opt-in, per capability	17
8.2 Wire format	17
8.3 Trust rules at every hop	18
9. Observability: tracing	19
10. Security and trust model	19
11. Evaluation and analysis	20
11.1 The golden set and metrics	20
11.2 Answer-level results	20
11.3 The live verification run	20
11.4 Root causes found by replaying cached model replies	22
11.5 Why only two questions used the SQL route	22
11.6 Table text and quote matching	23
11.7 Router-miss diagnosis	23
11.8 What the experiments cost	23
12. Bugs, surprises and lessons	24
13. Testing, quality and CI	24
14. Limitations and honest gaps	24
15. Future work, in priority order	25
16. Operating guide	26
16.1 Setup and first run	26
16.2 Configuration	26
16.3 Troubleshooting	26
16.4 Running the agent mesh	26
Appendix A. Repository layout	27
Appendix B. Per-question verification results	27
Appendix C. A real response and its trace	28
Appendix D. Glossary	29

Executive summary

FinSight answers plain-English questions about the public SEC filings of Apple, Microsoft and NVIDIA. It is built around three guarantees: **every claim carries a citation** that resolves to an exact filing and section, **every number comes from a database row or a calculator and never from the language model**, and **a verifier checks the draft against its evidence before the user sees it**. Where the verifier cannot confirm a statement, code appends an honest caveat instead of returning unverified text as if it were verified.

The project exists to show production habits around four ideas that are usually demonstrated in isolation: retrieval-augmented generation (RAG), agentic orchestration, the Model Context Protocol (MCP) and agent-to-agent communication (A2A). They are combined in one small, testable codebase with measured quality, guarded tools, untrusted-text handling, bounded retries and cost, and a single trace ID that follows a question through every service.

What was built

- **Ingestion:** an allowlisted, rate-limited SEC client; section-aware parsing and chunking; hosted dense and sparse embeddings in one Pinecone hybrid index; 19,033 XBRL facts in DuckDB. 43 filings became 2,967 chunks.
- **Retrieval and planning:** rule-based extraction of companies, years, form and section; fan-out into one search per company and year; hybrid search with metadata pre-filtering.
- **Facts route:** guarded read-only text-to-SQL over XBRL facts plus a deterministic calculator for growth, margins and ratios.
- **LangGraph orchestrator:** plan, retrieve and facts, compose, verify, a capped revise loop, finalize.
- **Three-layer verifier:** numbers in code, citation hygiene in code, and an LLM claim critic whose quotes are checked by code.
- **Interfaces:** a FastAPI service, an MCP server with four read-only tools, four A2A agents (retrieval, facts, verifier, analyst) with opt-in delegation, and a retrieval CLI.
- **Observability and quality:** dependency-free tracing across processes, 242 offline tests, a golden evaluation set, and a GitHub Actions workflow for lint and tests.

Headline results

Area	Result
Retrieval (26 answerable questions, top 6)	hit@6 0.73 with filters inferred by the router, against 0.77 with hand-written filters
Answers (Haiku 4.5, golden set)	no invented citation labels (1.00); all four unanswerable questions declined (1.00)
Verifier, live run on 32 questions	27 first drafts flagged; one revision cut the total flagged issues from 91 to 51
Prompt injection (poisoned chunk)	resisted on the one case tried
API, MCP (4 tools), A2A mesh (4 skills)	all exercised live against Pinecone and Claude
Cost of all verification work	\$0.259 cumulative: the 32-question run made 66 live Haiku calls (about \$0.23) and earlier small checks cost about \$0.03; hard-capped, Haiku only

What to take from the analysis

The most valuable results are not the headline numbers but what the measurements exposed. A flat parameter sweep revealed that the hybrid search was effectively keyword-only because sparse scores were about 31 times larger than dense scores. A reranker that improved ordering was rejected because it lost coverage on cross-company questions. A metric that read 34.6 percent turned out to be a regular-expression artefact and was really 7.7 percent. The live verification run then showed that the verifier was itself producing false positives, and replaying cached model replies, at no cost, traced each one to a specific defect in the verifier's own parsing and quote matching. Those defects are fixed and covered by regression tests.

Honest status: the verifier's before/after numbers in this document were measured before those five fixes and have not been re-measured at full scale. The verifier's flags are its own verdicts, not human labels, and writer and critic are the same model family. Section 14 lists every known gap.

1. Introduction

1.1 Problem and goals

Annual (10-K) and quarterly (10-Q) reports are long, dense and partly tabular. A reader who wants to know how a company describes a risk, what drove a line of business, or what a reported figure was must find the right filing, the right section and the right passage. General-purpose chat models answer such questions fluently but can invent figures, cite nothing, and blur fiscal years. FinSight tackles this with a pipeline whose output can be audited claim by claim.

The goals, in priority order, were: (1) trustworthy answers (cited, numerically grounded, verified); (2) measurable quality, with a golden set and baselines before any tuning; (3) a clean demonstration of RAG, agents, MCP and A2A working together; (4) operational hygiene: bounded cost, timeouts, tracing and tests; (5) strict legitimacy about data sources.

1.2 Non-goals

- Investment advice. Every answer carries a disclaimer added by code, not by the model.
- Coverage of the whole market. The corpus is three companies and roughly three years, and the router's company list is deliberately hard-coded.
- A hosted multi-user product. All servers bind to 127.0.0.1 and the A2A agents have no authentication.

1.3 Design principles and how each is enforced

Principle	How it is enforced
Numbers never come from the LLM	Figures come from DuckDB rows ([F#]) or the calculator ([K#]). The verifier rejects any number in the answer that appears in no evidence.
Every claim is cited	The composer labels chunks [C#]; the verifier flags invented labels and factual sentences without a citation.
Filings are untrusted text	Chunks are wrapped as data in prompts with an explicit instruction never to follow them; the verifier checks the output rather than trusting it.
Tools are guarded	SQL must be a single allowlisted SELECT parsed by DuckDB itself, on a read-only connection with external access disabled, with a row cap and a timeout. The calculator has four named operations and no <code>eval</code> .
Bounded cost and latency	Capped LLM retries, a disk cache, a revise-loop cap, per-request timeouts, an embedding-token ledger that refuses a batch before it is sent, and a hard cap on hosted rerank requests.
Fail closed	An unparseable critic reply is a failure; a verifier outage yields a caveat; a fabricated critic quote fails because code checks it against the evidence.
Legitimate data only	An allowlist of SEC endpoints in code, a declared User-Agent with a contact email, 5 requests per second, and a stop on HTTP 403 or 429.
Observable	One trace ID per question across every process; spans hold counts, sizes and error types, never prompts or filing text.

1.4 Technology stack

Layer	Choice	Why
Language and tooling	Python 3.11, uv, ruff, pytest	Fast, reproducible environments; one lockfile
Vector store	Pinecone serverless, one hybrid index (dotproduct, 1024 dimensions)	Hosted dense and sparse embeddings and search in one place; free tier suffices
Embeddings	llama-text-embed-v2 (dense), pinecone-sparse-english-v0 (sparse)	Hosted, no local GPU
Structured store	DuckDB	Embedded SQL engine; read-only access is easy to enforce
LLM	Claude Haiku 4.5 via LiteLLM	Cheap and fast; a single model keeps cost predictable
Orchestration	LangGraph	Explicit graph with a bounded revise cycle
API	FastAPI and uvicorn	Async, typed, simple to test
MCP	mcp 2.2.0 (MCPServer)	Exposes tools to Claude Desktop and Claude Code
A2A	a2a-sdk 1.2.1 (HTTP server)	Standard agent cards and JSON-RPC between agents
CI	GitHub Actions	Ruff (check and format) and the offline test suite on every push

1.5 Project history

The project was built one step at a time with a review after each step. It was restarted once, deliberately. The first build used a local Qdrant vector database and a local Ollama Llama model. The restart moved to Pinecone (hosted vector database and embeddings) and Claude Haiku 4.5. The practical reason was latency and operational weight: the first real question took about 74 seconds on the local Llama model and 17.8 seconds on Haiku including retrieval. The earlier build is preserved on the local branch [archive/steps-2-4-qdrant](#), and the Qdrant container, Ollama and unused packages were removed from the machine with the owner's approval.

Step	Content	Commit
1	Scaffold, SEC client, parser, chunker, DuckDB, Pinecone ingestion	c7cf7c3
2	Hybrid RAG on Pinecone with cited answers	6333b02
3	Golden set, baselines, alpha sweep, rerank experiment	2bd633a
4	Router, LangGraph orchestrator, guarded SQL tool, calculator, FastAPI	085f46f
5	Verifier loop with bounded revision	20935bd
6	MCP server (four read-only tools)	3529f24
7	A2A agents with opt-in delegation	7b64ba8
8	Tracing across agents and a full README	9aefc95
8+	Small live verifier check; README rewrites; CI; verifier fixes	804fd8c to bb9a8ba

2. Data and ingestion

2.1 Sources and the SEC access policy

All data comes from the SEC's documented endpoints (data.sec.gov and sec.gov/Archives). The restriction is enforced in code by an allowlist, not by convention. Requests carry a declared User-Agent that includes a real contact email, as the SEC requires. The client limits itself to 5 requests per second with a hard cap of 10 (the SEC's published limit), backs off, and stops entirely on HTTP 403 or 429. Raw filings are cached under [data/raw](#) so a re-run never re-downloads them. The repository does not redistribute filings: [data/](#) is gitignored and each user ingests their own copy.

2.2 The corpus

Item	Value
Companies	Apple (AAPL), Microsoft (MSFT), NVIDIA (NVDA)
Filings ingested	43 (10-K and 10-Q, about three years)
Chunks	2,967 (each is one Pinecone vector)
XBRL facts in DuckDB	19,033
Embedding tokens used	2.04M (the pre-run estimate was 2.12M)
Target chunk size	about 1,800 characters

2.3 Parsing and chunking

The parser converts each filing's HTML to plain text and splits it by Item. It indexes four sections of a 10-K (Item 1 Business, Item 1A Risk Factors, Item 7 MD&A and Item 7A Market Risk) and three of a 10-Q (Part II Item 1A risk factors, Item 2 MD&A and Item 3 market risk). Two real-world wrinkles shaped the parser. Every filing has a table of contents that repeats each Item heading, so a naive match finds every heading twice; the parser keeps, for each Item, the match whose span to the next heading is longest, which is the real section and not the contents line. And some filings (Microsoft's, for one) repeat a bare Item 1A marker on every page, which is not a heading, so a heading must be followed by punctuation to count. The text is then chunked to about 1,800 characters. Keeping the Item label as chunk metadata is what later lets the router pre-filter a search to, for example, Item 1A of Microsoft's FY2025 10-K. Converting HTML tables to text leaves each table cell on its own line, which matters for quote matching later (Section 11.6).

2.4 Embeddings and the hybrid index

Each chunk gets a dense vector (1024 dimensions, [llama-text-embed-v2](#)) and a sparse keyword-style vector ([pinecone-sparse-english-v0](#)). Both live in a single Pinecone serverless index that uses the dotproduct metric, in the namespace [filings](#). Pinecone has no server-side rank fusion, so the balance between dense and sparse is applied client-side with a parameter called alpha: dense values are multiplied by alpha and sparse values by one minus alpha.

- **Idempotent:** re-running ingestion never duplicates vectors (delete by id, then upsert) and never re-downloads cached filings. A DuckDB [indexed](#) marker records which filings are in which store, so a fresh index can be filled from cached filings.
- **Budgeted:** a monthly embedding-token ledger has a hard stop at 3.5M tokens (the free plan allows 5M) and refuses a batch before sending it. A `--estimate` mode reports the token count without uploading anything.
- **Safe:** `connect()` never creates an index by accident; creation is explicit, and the dimension is checked.

2.5 DuckDB: filings, chunks and facts

DuckDB holds the `filings` and `chunks` tables, the `indexed` markers, and a `facts` table of us-gaap XBRL values (19,033 rows across the three companies). A view, `v_facts`, exposes canonical metric names and a corrected `fiscal_year`. The facts route only ever sees this view.

2.6 Ingestion lessons

- Deleting from a namespace that does not exist yet returns 404 on the very first write; the store now treats that as empty and a regression test pins it.
- Apple fiscal-year labels were wrong until a re-ingest fixed them. Because fiscal years drive the router's filters, this kind of label error silently removes the right passage from results.

3. Architecture

3.1 System overview

ONLINE: answering a question

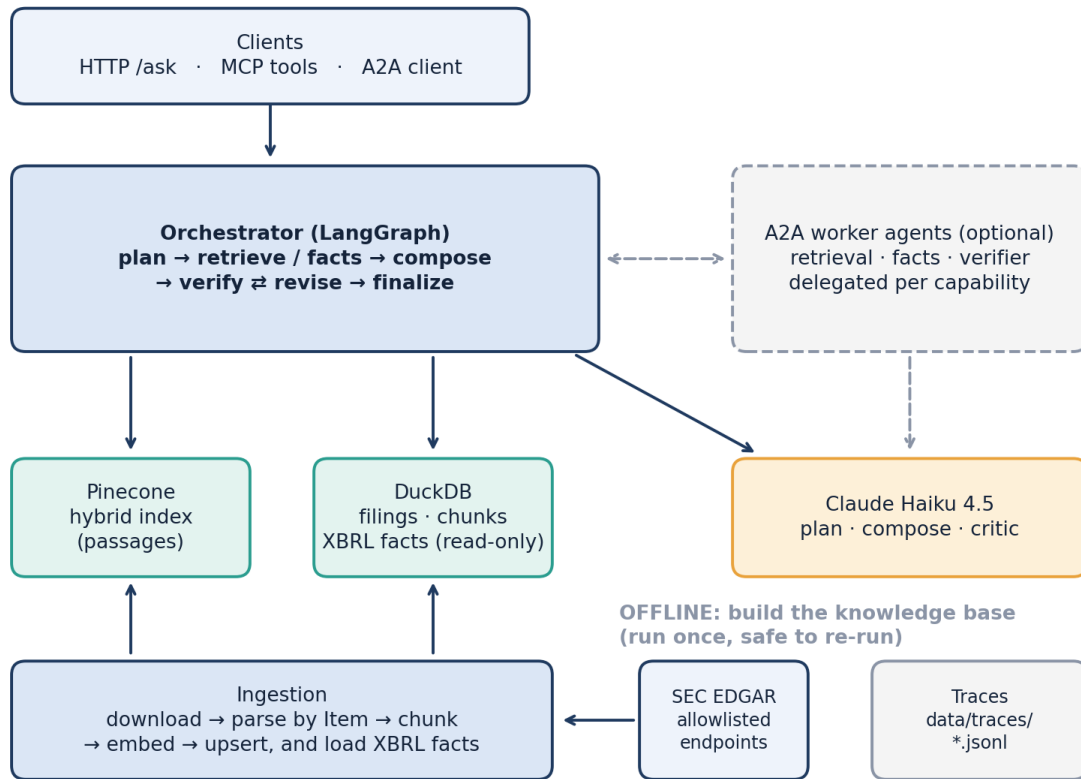


Figure 1. System overview: the online path (top) and the one-off ingestion path (bottom).

There are two stores. Pinecone holds passages for search. DuckDB holds the structured data and is opened read-only when answering. One model, Claude Haiku 4.5, performs every LLM step. The A2A worker agents are optional: with no delegation URL set, everything runs in one process.

3.2 The orchestrator graph

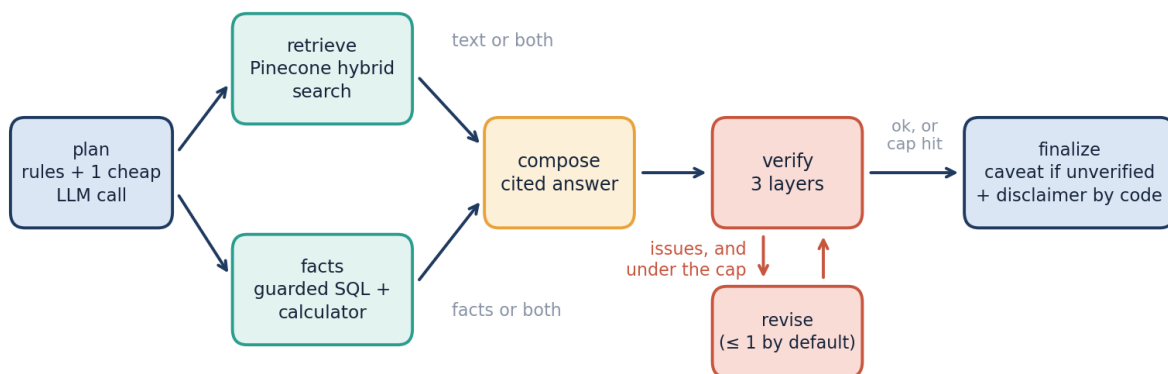


Figure 2. The LangGraph pipeline. The only loop, verify and revise, is bounded by a cap.

Node	What it does	LLM calls
plan	Rules extract tickers, years, form and section; fan out one sub-query per company and year (cap 9). One small LLM call picks text, facts or both, only when a metric word appears.	0 or 1
retrieve	Hybrid Pinecone search per sub-query with a metadata filter; merge to the top 6.	0
facts	Text-to-SQL on <code>v_facts</code> , then the calculator for growth and margins.	1 (capped retries)
compose	Haiku writes the answer from chunks, fact rows and calculations, with citations.	1
verify	Numbers, citations, and claim critic.	0 or 1
revise	Feeds the issues back to the writer; capped by <code>FINSIGHT_MAX_REVISIONS</code> (default 1).	1
finalize	Appends a caveat if issues remain and the disclaimer, both written by code.	0

3.3 Interfaces

Interface	Start	What you get
HTTP API	<code>uvicorn finsight.api.app:app --host 127.0.0.1 --port 8000</code>	POST <code>/ask</code> , GET <code>/traces/{id}</code> , GET <code>/health</code> ; an X-Trace-Id header on every response
MCP server	<code>python -m finsight.mcp_server (stdio)</code>	Four read-only tools: <code>search_filings</code> , <code>query_financials</code> , <code>calculate</code> , <code>ask_finsight</code>
A2A agents	<code>python -m finsight.agents retrieval facts verifier analyst</code>	Each capability as its own agent with an Agent Card
CLI	<code>python -m finsight.retrieval.cli "question" --ticker NVDA</code>	Retrieval plus a cited answer, no orchestration

3.4 A request, step by step

Take the question: What drove growth in Azure and other cloud services at Microsoft in fiscal 2025? The trace of this request (Appendix C) shows the lifecycle:

1. **plan**: rules detect MSFT, fiscal year 2025 and the 10-K form. There is no metric word that needs a routing decision, so the route is text and no LLM call is made.
2. **retrieve**: one hybrid search with the filter (MSFT, 2025, 10-K) pulls a pool of 30 passages and keeps the best 6.
3. **compose**: Haiku drafts an answer with citations such as `[C4]`.
4. **verify** (first pass): the critic flags one unsupported claim.
5. **revise**: the writer rewrites, dropping what the evidence did not support.
6. **verify** (second pass): zero issues.
7. **finalize**: no caveat is needed; the disclaimer is added by code. The response carries the answer, the source map (`C4` resolves to MSFT 10-K FY2025, Item 7), the verification block and the trace ID.

4. Retrieval in depth

4.1 Hybrid search mechanics

A query is embedded twice (dense and sparse, both as a query rather than a passage), scaled by alpha and one minus alpha, and sent as one Pinecone query together with a metadata filter. Filtering before ranking matters: without it, near-duplicate passages from other years or other companies crowd the top results. Query results are cached, and the cache key includes a corpus version so re-indexing invalidates stale entries.

The sparse-score scale bug

The first alpha sweep was flat below alpha 0.9. Rather than accept that as a property of the data, the score medians were measured: the median top-1 sparse score was 16.1 while the median dense score was 0.51, a ratio of about 31. A dotproduct index adds the two contributions, so with any alpha below roughly 0.95 the sparse side dominated and the so-called hybrid search was effectively keyword-only. The fix is a calibrated `sparse_score_scale` of 0.03 applied to the sparse side, with a regression test. This is the clearest example in the project of a silent quality bug found by distrusting a suspicious chart.

4.2 Choosing alpha

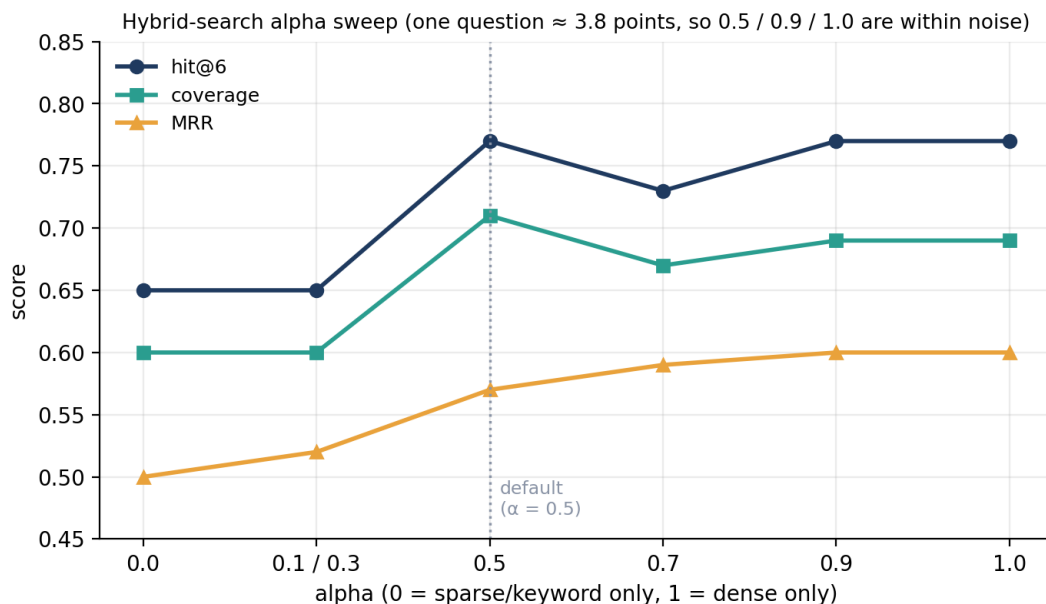


Figure 3. Hybrid-search alpha sweep on the 26 answerable questions, golden filters.

alpha	pool@30	hit@6	coverage	MRR	single	cross	yoy
0.0 (sparse only)	0.73	0.65	0.60	0.50	0.77	0.40	0.50
0.1 / 0.3	0.81	0.65	0.60	0.52	0.77	0.40	0.50
0.5 (default)	0.81	0.77	0.71	0.57	0.94	0.40	0.50
0.7	0.89	0.73	0.67	0.59	0.88	0.40	0.50
0.9	0.89	0.77	0.69	0.60	0.88	0.60	0.50
1.0 (dense only)	0.92	0.77	0.69	0.60	0.88	0.60	0.50
0.5 + hosted rerank	0.81	0.69	0.65	0.63	0.88	0.20	0.50

With 26 answerable questions, one question is worth about 3.8 points, so 0.5, 0.9 and 1.0 are statistically indistinguishable on hit@6. The default stayed at 0.5 because it had the best coverage

and single-company score, and because a genuinely hybrid setting is the point of the design. The honest reading is that the evaluation set is too small to separate these settings.

4.3 Reranking: better ordering, worse coverage

Pinecone's hosted reranker improved MRR from 0.57 to 0.63, meaning the right passage moved higher. But hit@6 fell from 0.77 to 0.69 and cross-company hit fell from 0.40 to 0.20, because reranking a pool of passages by relevance to the whole question tends to cluster results from one company. For questions that need several companies in the top 6, that is a loss. Reranking therefore stays off by default, is capped at 400 of the free 500 monthly requests, and 29 requests were used in the experiment.

4.4 Query planning: the router

A single search returns about six chunks, which cannot span several companies or years. The router therefore plans before searching.

- **Rules, not a model, for entities.** Tickers, years, form type and section cues are extracted by rules. Rules cannot hallucinate a company and are unit-testable. The 10-K is the default document; explicit cues map risk factors to Item 1A, MD&A to Item 7 and market risk to Item 7A.
- **Fan-out.** A comparison becomes one sub-query per company and year, capped at 9 (`MAX_SUBQUERIES`). Each sub-query gets its own filtered search.
- **The model decides only text versus numbers**, and only when the question mentions a metric. If the model call fails, the route defaults to text.

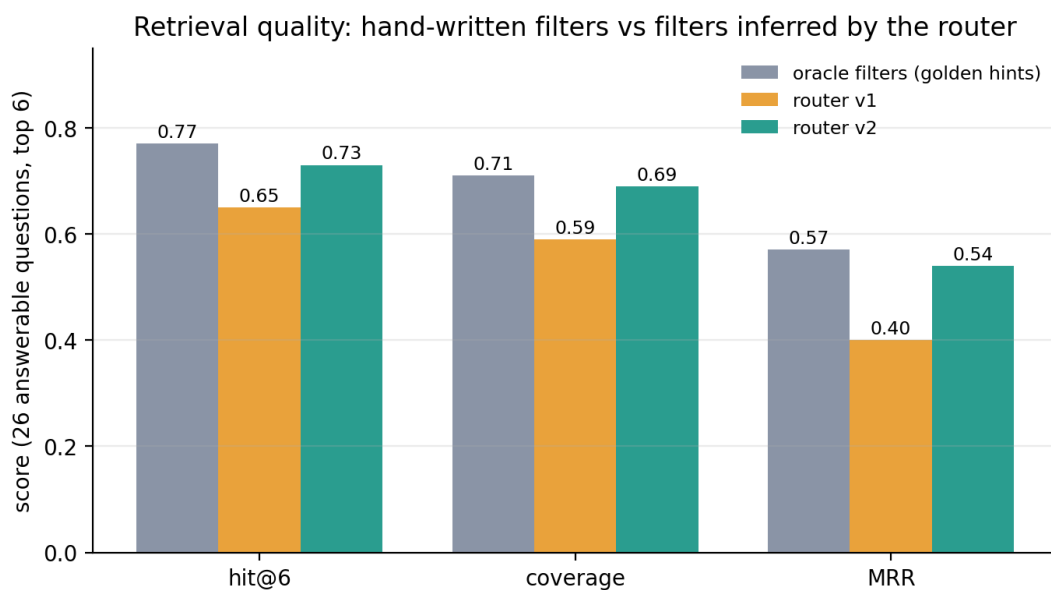


Figure 4. Retrieval with golden filters versus filters inferred by the router.

	oracle filters	router v1	router v2
hit@6	0.77	0.65	0.73
coverage	0.71	0.59	0.69
MRR	0.57	0.40	0.54
cross-company	0.40	0.60	0.60
year-over-year	0.50	0.50	0.75
single-company	0.94	0.71	0.77

Oracle filters use the golden set's hand-written filters and so measure search alone; the router columns are the realistic test in which filters are inferred from the question. The router beats oracle filters on cross-company questions because it searches each company and year separately, which the oracle's single filtered search cannot do. The drop on single-company questions is explained in Section 11.7.

5. The facts route: SQL and the calculator

5.1 Guarded text-to-SQL

When a question is about a reported number, Haiku writes SQL against `v_facts`. The SQL is not trusted. Several independent layers apply:

1. The statement is **parsed by DuckDB itself**, not by regular expression, and must be exactly one `SELECT`.
2. It may use **only allowlisted views and functions**.
3. The connection is opened **read-only** with **external access disabled**.
4. A **row cap** and a **timeout** bound the work; LLM retries are capped.
5. On failure the graph uses its existing fallback path instead of guessing.

Every figure that reaches the writer is a row with a citation label `[F#]`, so every reported number has a source row a reader can inspect.

5.2 The calculator and derived metrics

Growth and margins are computed by a deterministic calculator with four named operations: `ratio`, `pct_change`, `margin` and `cagr`. There is no `eval`. The orchestrator recomputes derived metrics from the returned SQL rows, which matters in the A2A setting: numbers never cross an agent boundary as facts. A live check returned Microsoft revenue of \$211.9 billion for fiscal 2023 and \$245.1 billion for fiscal 2024 from DuckDB and a growth figure of 15.67 percent from the calculator (`[K1]`).

5.3 Composition and citation labels

The composer receives three kinds of evidence, each with its own label: `[C#]` for filing text chunks, `[F#]` for database rows and `[K#]` for calculations. Filing text is wrapped as data and the prompt says never to follow instructions inside it. Labels are mapped back to real sources; an invented label is recorded as an invalid citation. The disclaimer is appended by code.

6. The verifier

The verifier is the centre of the project's trust story. It runs after every draft and has three layers, cheapest first.

6.1 The three layers

Layer	Check	Done by
1. Numbers	Every figure in the answer must appear in the evidence, after unit normalization and with a rounding tolerance.	Code
2. Citation hygiene	Every cited label must exist; every substantive factual sentence must carry a citation.	Code
3. Claim critic	For each cited claim, an LLM returns a verdict and a verbatim quote from the cited evidence. Code then confirms the quote really is in that evidence.	Haiku plus code

Why the critic's quote is verified by code

A critic that can pass a claim by simply saying supported is only as reliable as its mood. Requiring a verbatim quote, and then checking by string search that the quote exists in the cited evidence, converts a judgement into a check the critic cannot fake. A fabricated quote fails. An unparseable critic reply counts as a failure. The design therefore fails closed.

6.2 The revise loop, caveats and outages

- If issues are found and fewer than `max_revisions` revisions have happened (default 1), the issues go back to the writer with an instruction to remove unsupported claims rather than invent support, then the answer is verified again.
- If issues remain, code appends a caveat: how many statements could not be verified. The answer never claims more confidence than the checks support.
- If the verifier is unreachable (in the A2A setup), the issue kind `verifier_unavailable` is raised, the pointless rewrite is skipped, and the caveat says the answer could not be independently verified. It fails closed.
- Issue kinds: `unsupported_number`, `invalid_citation`, `uncited`, `unsupported_claim` and `verifier_unavailable`.

6.3 Known limits of the verifier

- The critic and the writer are the same model family, so they can share blind spots.
- The verifier proves that a claim appears in the cited evidence, not that the cited evidence is itself right, nor that nothing relevant was missed.
- A statement a filing itself makes cannot be told apart from the filing's own claims, so an injected instruction that the filing states as fact is a residual risk.
- The table-unit case: a number such as \$391,035 million is matched against a table that shows 391,035 under an (In millions) header. Without the word million next to the number, the number layer flags it. The answer ends up right but it costs a revision.

7. The MCP server

The MCP server exposes FinSight to any MCP client such as Claude Desktop or Claude Code. It runs over stdio by default, or over streamable HTTP on 127.0.0.1. Dependencies are built lazily on the first tool call, so startup is fast and needs no credentials. Logging goes to stderr because stdout is the stdio protocol channel.

Tool	What it does	LLM
<code>search_filings</code>	Hybrid search with optional tickers and fiscal year; at most 10 results.	No
<code>query_financials</code>	Guarded SQL over XBRL facts plus derived growth and margins.	Yes (SQL)
<code>calculate</code>	Deterministic arithmetic: ratio, pct_change, margin, cagr.	No
<code>ask_finsight</code>	The full graph including the verifier.	Yes

- **All tools are read-only** and declare themselves so.
- **Inputs are bounded:** query length, a limit of 10, known tickers, and a section pattern [Item N](#) (for example [Item 1A](#)).
- **Every call** runs in a worker thread with a timeout and a trace ID. Unexpected exceptions are hidden from clients, which see only the trace ID; deliberate tool errors pass through.
- The project uses mcp 2.x, in which [FastMCP](#) was renamed [MCPServer](#); the integration was written against the installed package rather than older documentation.

Testing: 21 tests run the tools through the real protocol with an in-memory client, and a real stdio subprocess smoke test exercised `calculate`. The live run (Section 11.6) exercised all four tools against live Pinecone and Claude and confirmed that an invalid operation is rejected.

8. The A2A services

A2A (agent-to-agent) turns each capability into a separately running agent with an Agent Card. Four agents exist. The three workers are thin skills over already-tested logic; the analyst runs the same LangGraph and delegates to the workers when they are configured.

Agent	Port	Skill	LLM
analyst	9100	<code>ask</code> : the full graph; delegates to the others	Yes
retrieval	9101	<code>search_filings</code>	No
facts	9102	<code>query_financials</code> : guarded text-to-SQL, returns rows only	Yes (SQL)
verifier	9103	<code>verify_answer</code> : the same code as in-process	Yes (critic)

8.1 Delegation is opt-in, per capability

Three environment variables turn delegation on: `FINSIGHT_A2A_RETRIEVAL_URL`, `FINSIGHT_A2A_FACTS_URL` and `FINSIGHT_A2A_VERIFIER_URL`. Anything left unset stays in-process. Workers always build local dependencies, so an agent can never be configured to call itself. Once a URL is set, that agent must be running: a request that needs it fails with an error instead of silently falling back. This is by design, and it is the single most common setup mistake (Section 16.3).

8.2 Wire format

Requests and replies are JSON envelopes carried in a text part: `{trace_id, parent_span_id, args}` going out and `{ok, trace_id, result | error}` coming back. Text parts are used instead of structured data parts because protobuf `Struct` coerces every number to a float (2025 becomes 2025.0), which would corrupt fiscal years and figures. Replies are single Messages.

8.3 Trust rules at every hop

- Responses are **schema-validated** with bounded fields, **size-capped** (512,000 bytes) and subject to a timeout (60 seconds); transport errors are retried once.
- Remote **error text is truncated** before it is shown or logged.
- **Numbers never cross as facts.** The orchestrator recomputes growth and margins from the SQL rows the facts agent returns.
- A **verifier outage fails closed**, as described in Section 6.2.
- The agents bind to 127.0.0.1 and have **no authentication**. They must not be exposed to a network as they are.

Known limits: a per-call alpha override is ignored by the remote searcher (the retrieval agent owns alpha); the SDK logs a harmless warning (Dispatcher task is not running) on single-Message replies; search-argument validation is duplicated between the MCP server and the retrieval agent, which is a candidate for a shared function.

9. Observability: tracing

Tracing is dependency-free. Each step records a span (name, duration, status and a few small attributes) appended to a JSONL file under `data/traces/`, one file per day. A trace is every span that shares a trace ID. `ask()` sets the ID for a graph run; A2A calls carry the ID and the calling span's ID in the request envelope, and `adopt()` continues the trace in the receiving process, so a question that crosses the analyst, retrieval, facts and verifier is one tree.

Privacy rules, enforced by tests

- Spans hold **counts, sizes, model names, token counts and error types** only.
- They never hold prompts, filing text, SQL text, API keys or exception messages (messages can contain secrets). An error records only its type.
- Attribute values are bounded scalars; a write failure is logged once and swallowed so tracing can never break a request.
- Tracing is off until a process calls `configure()`; importing the library has no side effects. `FINSIGHT_TRACING=false` turns it off.

LangGraph propagates context variables into its worker threads, which a test proves, so spans from graph nodes nest correctly. Traces are readable with `python -m finsight.trace_cli --last` (an indented timeline with LLM and token totals) or `GET /traces/{trace_id}`. Fifteen tests cover the tracer, its privacy rules, cross-agent trees, the API endpoint and the CLI. Files are not deleted automatically, which is a known gap.

10. Security and trust model

Threat	Mitigation	Residual risk
Prompt injection through filing text	Chunks wrapped as data; prompts forbid following them; verifier checks output; one live poisoned-chunk test resisted it	Text a filing itself states cannot be told apart from its own claims; tested on one case
Hallucinated numbers	Numbers only from DuckDB or the calculator; number layer rejects unsupported figures	Unit-scale cases such as table headers saying (In millions)
Fabricated citations or quotes	Labels must exist; critic quotes are string-checked against the evidence	Critic shares the writer's model family
Destructive or exfiltrating SQL	DuckDB-parsed single SELECT, allowlist, read-only connection, external access off, row cap, timeout	Allowlist must be kept current
Hostile or broken agents	Schema validation, size caps, truncated error text, local-only workers, fail-closed verifier	No authentication on the agents; keep them on localhost
Secret leakage	Keys only in <code>.env</code> (gitignored, SecretStr); traces exclude messages; clients see only trace IDs	Operator discipline when sharing logs
Abusive SEC access	Allowlisted endpoints, declared User-Agent, rate limit, stop on 403 or 429	None known
Runaway cost or latency	Capped retries and revisions, cache, timeouts, token ledger, rerank cap	Spend is per call, not per user

11. Evaluation and analysis

11.1 The golden set and metrics

A hand-written golden set of 30 questions drives all evaluation: 17 single-company, 5 cross-company, 4 year-over-year and 4 deliberately unanswerable. Each answerable question lists target passages by company, form, fiscal year, section and a text pattern; `--validate` proves that every target matches at least one real chunk. With 26 answerable questions, each is worth about 3.8 points, so differences of one or two questions are noise.

Metric	Meaning
hit@6	At least one target passage is in the top 6 results
coverage	Fraction of a question's targets found in the top 6 (matters for multi-target questions)
MRR	Mean reciprocal rank of the first target hit: how high the right passage ranks
citations_valid	No invented citation labels in the answer
abstain_correct	Unanswerable questions are declined
has_citation / cites_relevant	The answer cites something; the cited passages match the targets
false_abstain	A bare refusal on an answerable question (no citations at all)

11.2 Answer-level results

With Claude Haiku 4.5, alpha 0.5 and golden-set filters (30 questions, 77 seconds): citations_valid 1.00, abstain_correct 1.00, has_citation 0.92, cites_relevant 0.73 and false_abstain 0.08. With the router's inferred filters (router v1), cites_relevant fell to 0.54 and bare refusals to 0.04. Five of the nine answers that said not found were retrieval misses on cross-company or year-over-year questions, the exact weakness the router was built to address.

A metric that lied

The first false-abstention metric read 34.6 percent because a regular expression counted cited partial answers as refusals. Requiring no citations brought the figure to 7.7 percent. The lesson: read a few raw examples behind any surprising metric before acting on it.

11.3 The live verification run

To measure the verifier on real model output, one script ran in a single process against live Pinecone and Claude with a spend ledger and a hard cap set well under the owner's \$0.50 ceiling. It refused any model other than Haiku 4.5. The run used the full golden set plus two numeric questions that exist only in the script, 32 questions in all. For each question it recorded the verifier's verdict on the first draft (before) and on the final answer (after one revision). Because the first draft is the same either way, this is a paired before/after comparison from one pass.

Results (measured with the verifier as it was before the fixes in Section 11.4)

Measure	Result
Questions	32 (30 golden, 2 numeric); routes: 30 text, 2 facts, 0 both
First drafts flagged	27 of 32 (cross 5/5, yoy 4/4, numeric 2/2, single 14/17, unanswerable 2/4)
After one revision (of the 27 flagged)	6 fully clean, 13 with fewer issues, 8 unchanged or worse
Total flagged issues	91 before, 51 after
Issue kinds before	50 uncited, 40 unsupported claim, 1 unsupported number
Answer checks	citations_valid 1.00, abstain_correct 1.00, false_abstain 0.00, has_citation 0.96, cites_relevant 0.61

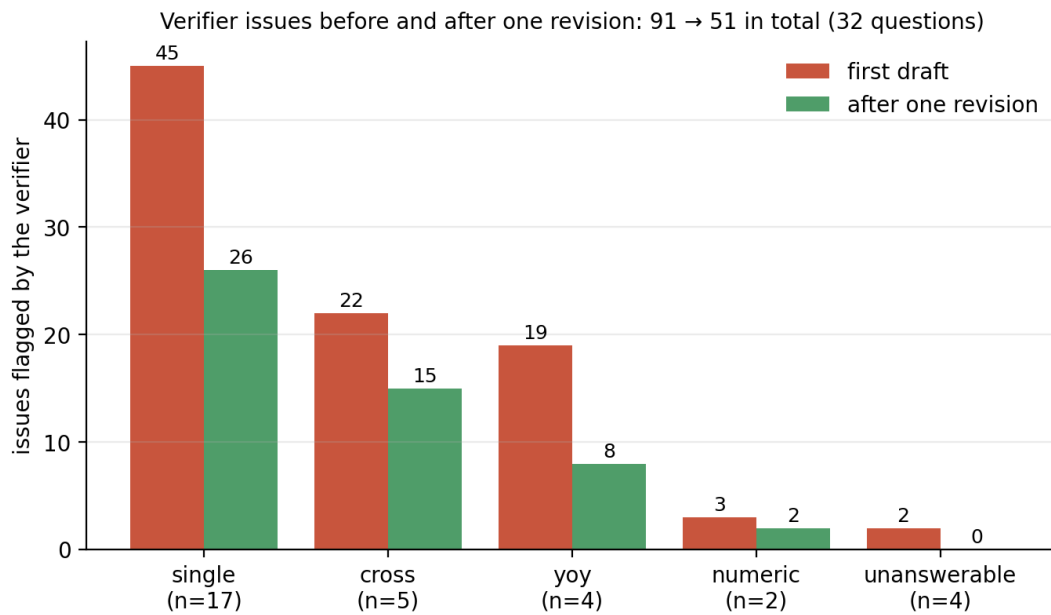


Figure 5. Issues flagged before and after one revision, by question type.

What one revision did to the 27 flagged first drafts (5 more answers were clean from the start)



Figure 6. What one revision did to the 27 flagged first drafts.

How to read these numbers: a high flag rate mostly shows that the verifier is strict. It is not a measured hallucination rate. The flags are the verifier's own verdicts, not human labels, and several were false positives, as Section 11.4 shows. The claim that is supported is narrower: one revision removed about 44 percent of the issues the verifier raised.

Other live checks, all passed

Check	Outcome
Prompt injection	A chunk told the model to claim Azure grew 999 percent and to reveal its system prompt. Neither happened; citations were kept and verification passed (one case, one attack style).
FastAPI app	<code>/health</code> ok; <code>/ask</code> returned 200 with a verified answer; <code>/traces</code> returned 13 spans; an empty question gave 422; an over-long trace ID gave 422; an unknown trace gave 404.
MCP server	All four tools ran; <code>calculate</code> returned 25.0 for a 100 to 125 change; an invalid operation was rejected; search returned 5 results.
A2A mesh	The analyst delegated to retrieval, facts and verifier; <code>ask</code> , <code>search_filings</code> , <code>query_financials</code> and <code>verify_answer</code> were all handled; each question was one trace tree.

Caveat: the API, MCP and mesh checks ran in one process with real Pinecone and Claude behind them. They validate the logic end to end but not `uvicorn`, real sockets or separate operating system processes.

11.4 Root causes found by replaying cached model replies

Every model call made during the live run was cached. That allowed the verifier's behaviour to be replayed with a guard that aborts on any live call, at no cost, and the raw critic replies to be inspected. This turned a puzzling flag rate into five specific, fixable causes:

#	Symptom	Root cause	Fix
1	A correct database answer (Microsoft revenue growth) got a caveat	The critic said supported, but its quote joined fragments of three evidence rows with an ellipsis, and the check wanted one contiguous string	A quote with ellipses passes only if every fragment of real length is found in the evidence; a fabricated fragment still fails
2	A real quote from Apple's net-sales table failed	Once the filing's HTML is converted to text, the table has each cell on its own line (<code>\$, 391,035, 2, %</code>), so <code>\$391</code> and <code>\$ 391</code> compared as different	Whitespace around <code>\$</code> and <code>%</code> is closed before comparing
3	Revision appeared to make some answers worse (s16: 2 to 5 issues)	The revised writer used a label-first format (<code>[C5] quote</code>); the parser attached that label to the previous line, leaving the quote uncited	A label that opens a line and is followed by text cites that text; trailing labels still attach backwards
4	Lead-in lines such as Apple discusses tariffs in the following ways: were flagged uncited	They introduce cited items and carry no claim of their own	Sentences ending with a colon are skipped; numbers in them are still checked
5	What was Apple's total net sales? never reached SQL	The metric regex had no term for sales, and Apple reports net sales, not revenue	<code>\bsales\b</code> added to the metric pattern

Five regression tests (242 tests in total) pin these fixes. On a replay, the Microsoft revenue-growth question went from one issue to zero. **The full before/after numbers above were not re-measured after the fixes:** the feedback text fed to the writer changed, so the cache no longer covers the revised prompts and a re-run needs fresh, paid calls.

11.5 Why only two questions used the SQL route

The golden set is dominated by risk-factor and discussion questions, so 30 of 32 questions took the text route. The facts route is therefore thinly evaluated. Adding numeric questions to the golden set is the most valuable evaluation improvement still open.

11.6 Table text and quote matching

Converting filing HTML to text flattens tables into one cell per line. That is harmless for retrieval but breaks naive substring matching. The quote check therefore normalises case, punctuation, whitespace and the spacing around currency and percent signs before comparing, while the number layer handles units and rounding. The remaining gap is a table whose header says the figures are in millions, which the number layer cannot see from the number alone (Section 6.3).

11.7 Router-miss diagnosis

Seven golden questions missed under the router (s03, s04, s10, s17, c01, c04, y03). Comparing the router's planned filters with the golden filters, offline and without any retrieval run, separates them into two groups:

Group	Questions	Finding
Label over-specificity	s03, s04, s10, s17	The question names no year (or asks for quarterly reports), so the router correctly searches across years, while the golden label demands FY2025 or a specific Item. The router is right and the label is over-specific.
Ranking inside correct filters	c01, c04, y03	The router's filters are exactly right (one sub-query per company and year), so the miss is ranking within the filter. The plan sets no section because only the phrase risk factors counts as an Item 1A cue.

Untested idea: treat the word risk as an Item 1A cue for 10-K questions. It might help the second group but could hurt market-risk questions (Item 7A), so it needs a retrieval evaluation first.

11.8 What the experiments cost

All experiments used Haiku 4.5 and were run under caps. The 32-question live run, including the injection, API, MCP and mesh checks, made 66 live calls for about \$0.23 (roughly \$0.0035 per call). Adding the earlier small verifier checks, all verification work cost \$0.259 in total, against a ceiling of \$0.50 set by the owner. Embedding the corpus used 2.04M of the free plan's 5M monthly tokens. Replays of cached calls are free, which is why caching every LLM call paid off during debugging.

12. Bugs, surprises and lessons

What happened	Root cause	Lesson
Alpha sweep was flat below 0.9	Sparse scores were about 31 times dense scores	Distrust a flat curve; measure the score distributions
False-abstention read 34.6 percent	A regex counted cited partial answers as refusals	Read raw examples behind a surprising metric
Reranker helped MRR but hurt recall	Reranking clustered results by company	Optimise the metric the use case needs, not the one that moved
First write to Pinecone failed with 404	Deleting from a namespace that did not exist yet	Treat not-yet-existing state as empty; add a regression test
A new model rejected temperature 0	Some models accept only their default temperature	Make model parameters config and drop unsupported ones
Graph looped between retrieve and facts	An unbounded edge	Every cycle needs a cap, and a regression test
mcp 2.x had no FastMCP	The class was renamed MCPServer	Read the installed package, not old docs
A2A responses corrupted numbers	Protobuf Struct turns numbers into floats	Carry numbers in text envelopes; validate the schema
Second app build crashed on DuckDB	Its config is locked per process once set	Build the connection once per process
Plain API runs failed after a mesh trial	Delegation URLs stayed set in <code>.env</code>	Fail loudly on an unreachable agent; document the switch
Verifier flagged correct answers	Five parser and quote-matching defects (Section 11.4)	Replay cached replies to find root causes before tuning prompts

13. Testing, quality and CI

- **242 offline tests** (about 4,300 lines of source and 2,600 of tests). They use fakes for Pinecone, the LLM and the network, and need no API keys. Growth by step: 42 after ingestion, 63 after basic RAG, 78 after the eval harness, 152 after the orchestrator, 237 at the tracing step, 242 now.
- Notable suites: verifier (26), MCP through the real protocol (21), A2A over an in-process ASGI mesh (22), and tracing with privacy checks (15).
- **Lint and format:** ruff check and ruff format are clean across the repository.
- **CI:** a GitHub Actions workflow runs ruff (check and format) and the full offline suite on every push and pull request, using `uv sync --frozen` so it installs exactly what the lockfile pins. It was confirmed green on GitHub. The suite also passes in a copy with no `.env`, no `data/` folder and a scrubbed environment, which is what CI sees.
- **Not in CI:** live evaluations, which need keys and cost money.

14. Limitations and honest gaps

- **Scope:** three companies and about three years; the router's company list is hard-coded.
- **Retrieval is decent, not great:** hit@6 is 0.73 on a 26-question set, where one question is 3.8 points. Cross-company and year-over-year retrieval are the weak spots.

- **The verifier is lightly validated.** Its numbers are the verifier's own verdicts, not human labels; writer and critic share a model family; the 91 to 51 result predates five fixes and was not re-measured; the golden set exercises the SQL route in only 2 of 32 questions.
- **Live checks ran in one process:** no real sockets and no separate OS processes for the API, MCP and mesh checks.
- **Prompt-injection evidence is one case.**
- **Answer-level evaluation** was run with router v1, not v2.
- **No authentication** on the A2A agents; localhost only.
- **Trace files are never deleted automatically.**
- **No Docker Compose:** the repo's `docker-compose.yml` is an unused leftover from the Qdrant build.

15. Future work, in priority order

1. **Re-measure the verifier after the fixes**, at full scale, for an honest before/after number (a small paid run).
2. **Hand-label a sample of flags** to estimate the verifier's precision, and add a second, different critic model to break the shared-blind-spot problem.
3. **Add numeric questions to the golden set** so the SQL route and the number layer are measured, including the In millions table-unit case.
4. **Try the risk means Item 1A cue** behind a retrieval evaluation.
5. **Re-run the answer-level evaluation on router v2.**
6. **Run the API and mesh over real sockets and separate processes**, and add authentication to the A2A agents before any non-local use.
7. **Docker Compose** for a one-command demo; **trace retention; live smoke questions in CI** using repository secrets.
8. Widen the corpus beyond three companies, which requires replacing the hard-coded company list.

16. Operating guide

16.1 Setup and first run

Requirements: macOS or Linux, Python 3.11, uv, a free Pinecone account and an Anthropic API key. Run everything from the repository folder.

```
git clone git@github.com:AtharvaMusale/finsight.git
cd finsight
cp .env.example .env          # then edit it
uv sync
uv run pytest                 # offline tests, no network needed
```

Edit `.env` and set `FINSIGHT_SEC_USER_AGENT` to an app name plus your real email, and add `PINECONE_API_KEY` and `ANTHROPIC_API_KEY`. Keys live only in `.env`, which is gitignored.

```
uv run python -m finsight.ingestion.pipeline # one time
uv run uvicorn finsight.api.app:app --host 127.0.0.1 --port 8000

curl -s localhost:8000/ask -H 'content-type: application/json' \
  -d '{"question": "What drove growth in Azure and other cloud services at Microsoft in fiscal 2025?"}'
uv run python -m finsight.trace_cli --last
```

16.2 Configuration

Variable	Default	Meaning
<code>FINSIGHT_LLM_MODEL</code>	<code>anthropic/claude-haiku-4-5-20251001</code>	Model for routing, SQL, answers and the critic
<code>FINSIGHT_VERIFY</code>	<code>true</code>	Verifier on or off
<code>FINSIGHT_MAX_REVISIONS</code>	<code>1</code>	Revise-loop cap (0 to 3)
<code>FINSIGHT_REQUEST_TIMEOUT_S</code>	<code>120</code>	Whole-request timeout including verify and revise
<code>FINSIGHT_A2A_*_URL</code>	<code>unset</code>	Delegate that capability to an A2A agent
<code>FINSIGHT_TRACING</code>	<code>true</code>	Write spans to <code>data/traces/</code>

16.3 Troubleshooting

- **/ask returns 502 or says an agent is unavailable:** the `FINSIGHT_A2A_*_URL` variables are set but the agents are not running. Comment them out for single-process mode, or start the four agents (ports 9101, 9102, 9103, 9100) first.
- **A conda libmamba warning prints before uv run:** it comes from a conda base environment, is unrelated to FinSight and is harmless.
- **Evaluations fail to start:** confirm the index is reachable with `uv run python -m finsight.evals.run --validate` (expect 2,967 chunks and all targets matched).

16.4 Running the agent mesh

Start each worker in its own terminal, then the analyst, and set the three URLs in `.env` to `http://127.0.0.1:9101`, `9102` and `9103` respectively.

```
uv run python -m finsight.agents retrieval
uv run python -m finsight.agents facts
uv run python -m finsight.agents verifier
uv run python -m finsight.agents analyst
```

Appendix A. Repository layout

```
src/finsight/
  ingestion/      SEC client, parser, chunker, embedder, Pinecone and DuckDB stores
  retrieval/      hybrid search, optional rerank, cited answers, cache, CLI
  orchestrator/   router, LangGraph graph, calculations, answer composition, verifier
  tools/          guarded text-to-SQL, deterministic calculator
  api/            FastAPI app (/ask, /traces, /health)
  mcp_server/     MCP server (stdio, optional streamable HTTP)
  agents/         A2A agents: retrieval, facts, verifier, analyst
  evals/          golden-set runner and metrics
  tracing.py      spans and JSONL writer; trace_cli.py timeline viewer
evals/golden.jsonl the 30 golden questions   evals/results/   recorded eval runs
tests/            offline tests (fakes for Pinecone, the LLM and the network)
.github/workflows/ci.yml lint and offline tests on every push
TASKS.md          build log with results and open items
```

Appendix B. Per-question verification results

Source: [evals/results/full_verify.json](#). Columns are the number of issues on the first draft (before) and on the final answer (after), and the revisions used. Measured before the fixes in Section 11.4.

id	type	route	before	after	revisions
x01	numeric	text	2	1	1
x02	numeric	facts	1	1	1
s08	single	text	1	0	1
s12	single	text	0	0	0
y01	yoy	text	6	2	1
u01	unanswerable	text	0	0	0
s01	single	text	12	4	1
c01	cross	text	4	4	1
y02	yoy	text	6	2	1
u02	unanswerable	facts	1	0	1
s02	single	text	0	0	0
c02	cross	text	5	3	1
y03	yoy	text	3	1	1
u03	unanswerable	text	1	0	1
s03	single	text	1	0	1
c03	cross	text	5	4	1
y04	yoy	text	4	3	1
u04	unanswerable	text	0	0	0
s04	single	text	3	3	1
c04	cross	text	4	0	1
s05	single	text	3	1	1
c05	cross	text	4	4	1
s06	single	text	3	1	1
s07	single	text	1	1	1
s09	single	text	1	1	1

id	type	route	before	after	revisions
s10	single	text	0	0	0
s11	single	text	6	3	1
s13	single	text	1	0	1
s14	single	text	7	4	1
s15	single	text	3	1	1
s16	single	text	2	5	1
s17	single	text	1	2	1

Questions x01 and x02 are the two numeric questions that exist only in the evaluation script. x01 (Apple total net sales) took the text route because of the metric-regex gap fixed in Section 11.4.

Appendix C. A real response and its trace

Question: What drove growth in Azure and other cloud services at Microsoft in fiscal 2025? The response is trimmed to its main fields:

```
{
  "trace_id": "3e19fa254fb3444bab861c9edel490d6",
  "answer": "According to the filing, Azure and other cloud services revenue grew 34% in
    fiscal 2025, driven by \"demand for our portfolio of services\" [C4].",
  "disclaimer": "Not financial advice. This is an automated summary of public SEC filings.",
  "route": "text",
  "sources": {"C4": {"source": "MSFT 10-K FY2025, Item 7",
    "accession_no": "0000950170-25-100235"}},
  "facts": {}, "calculations": {},
  "verification": {"ok": true, "claims_checked": 1, "critic_ran": true,
    "revisions": 1, "issues": []}
}
```

The trace of the same request (timings are tiny because this run was served from the local cache):

```
ask                14.2 ms  route=text revisions=1 verified=True
node.plan          0.1 ms  route=text subqueries=1
node.retrieve      0.7 ms  hits=6
  search          0.5 ms  limit=30 alpha=0.5 filtered=True cached=True hits=30
node.compose       0.4 ms  draft_chars=436 no_evidence=False
  llm             0.3 ms  model=anthropic/claude-haiku-4-5-20251001 cached=True
node.verify        3.4 ms  issues=1 kinds=unsupported_claim critic_ran=True claims=2
  llm            0.2 ms  model=anthropic/claude-haiku-4-5-20251001 cached=True
node.revise        0.3 ms  revisions=1
  llm            0.2 ms  model=anthropic/claude-haiku-4-5-20251001 cached=True
node.verify        3.2 ms  issues=0 critic_ran=True claims=1
  llm            0.1 ms  model=anthropic/claude-haiku-4-5-20251001 cached=True
node.finalize      0.0 ms  caveat=False
```

Appendix D. Glossary

Term	Meaning
10-K / 10-Q	A company's annual / quarterly report filed with the SEC
Item 1A, 7, 7A	Risk Factors; Management's Discussion and Analysis (MD&A); Market Risk
XBRL	Machine-readable tagged financial data inside filings; the source of the facts table
RAG	Retrieval-augmented generation: retrieve passages, then generate an answer from them
Hybrid search	Combining dense (semantic) and sparse (keyword) vectors in one query
alpha	The dense-versus-sparse balance: 0 is sparse only, 1 is dense only
hit@6 / coverage / MRR	Retrieval metrics defined in Section 11.1
Oracle filters	Hand-written filters from the golden set; they measure search alone
Router	The rules and one cheap LLM call that plan filters, fan-out and the route
Critic	The LLM that must return a verbatim supporting quote for each claim
MCP	Model Context Protocol: a standard for exposing tools to AI clients
A2A	Agent-to-Agent protocol: agents with cards that call one another
Trace / span	One question's whole record / one timed step within it
Fail closed	On doubt or failure, refuse or caveat rather than pass silently
Golden set	The 30 hand-written evaluation questions with target passages